An Empirical Study of GPU Kernel Performance Gaps in Modern Domain-Specific Languages

Anonymous Author(s)

Abstract

GPU kernel development is shifting from hand-written CUDA to domain-specific languages (DSLs) such as Triton and TileLang, which promise near-library performance with dramatically lower development effort. While DSLs demonstrably match or approach cuBLAS throughput on matrix multiplication workloads, practitioners report significant underperformance on convolution kernels [19]. This gap has been observed anecdotally but never systematically characterized.

We present the first empirical study of the performance gap between DSL-written kernels and optimized libraries (cuBLAS, cuDNN) across a representative suite of 21 GPU kernels drawn from TritonBench [10] and augmented with TileLang implementations, spanning five kernel categories: GEMM, attention, convolution, normalization, and element-wise operations. Our study characterizes the magnitude and distribution of the performance gap (RQ1), identifies root causes through latency-driven analysis and compiler-level reasoning (RQ2), and evaluates mitigations that narrow or close the gap on the most affected kernel categories (RQ3). Triton achieves a median library efficiency (ratio of library baseline latency to DSL latency, expressed as a percentage; 100% = parity) of 32–58% on GEMM depending on shape and 103% on element-wise kernels, but only 35% on convolution. TileLang, while competitive with cuBLAS on select large GEMM shapes (up to 56%), achieves less than 10% of cuDNN throughput on convolution. TileLang also exhibits a catastrophic normalization anomaly: 314× slower than PyTorch on large LayerNorm, consistent with sequential thread synchronization in its reduction primitives and absent vectorized memory loads—a root cause that hardware performance counter profiling is designed to confirm. Our analysis identifies four compiler-level root causes of convolution underperformance: absent vectorization of strided memory accesses (RC1), auto-tuning search space mismatch (RC2), register pressure from large-filter accumulations (RC3), and missing Winograd algorithm support (RC4). For five representative kernels, targeted fixes address each root cause directly: correcting TileLang's reduction primitive usage brings LayerNorm and RMSNorm to 98% of PyTorch throughput (recovering from a 314× latency deficit); restructuring Conv2d as an implicit GEMM with alignment and extended auto-tuning narrows the Triton convolution gap to 80% of cuDNN efficiency, with the remaining 20% attributed to absent Winograd algorithm support and identified as future compiler work. Our kernel suite, profiling scripts, and raw results are publicly available to support reproducibility.

Keywords

GPU kernels, domain-specific languages, Triton, TileLang, cuBLAS, cuDNN, empirical study, performance analysis, convolution

ACM Reference Format:
Anonymous Author(s). 2026. An Empirical Study of GPU Kernel Performance Gaps in Modern Domain-Specific Languages. In . ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Modern deep learning workloads run on GPUs, and their performance is determined largely by the efficiency of a small number of computational kernels: matrix multiplications, attention operations, and convolutions. Historically, these kernels were provided by hand-optimized closed-source libraries: cuBLAS for dense linear algebra and cuDNN for deep learning primitives [4, 12]. PyTorch, TensorFlow, and other frameworks delegate to these libraries automatically, so practitioners have rarely needed to write GPU code directly.

This is changing. The rapid expansion of model architectures — sparse attention, mixture-of-experts routing, custom activation functions, fused decode kernels — has outpaced the coverage of static libraries. Domain-specific languages (DSLs) such as Triton [18] and TileLang [21] have emerged as the primary tools for writing custom GPU kernels without CUDA expertise. Triton, in particular, is now the default backend for `torch.compile` [1]: any operator that PyTorch cannot fuse into a library call is lowered to a Triton kernel automatically.

For GEMM-like kernels, DSLs perform well. Triton's original 2019 paper demonstrated a matrix-multiply kernel competitive with cuBLAS in under 25 lines of Python. FlashAttention [6] and its successors, written in Triton, have become the standard attention implementation across the industry. These successes have reinforced the perception that DSLs offer a free productivity-performance tradeoff.

For convolution kernels, the picture is different. Since at least 2022, practitioners have reported in the Triton project's public issue tracker that Triton-written convolutions fall substantially short of cuDNN performance on ResNet-scale workloads [19]. The root cause has been identified informally — absent vectorization and cascade effects on asynchronous memory copies — but no systematic study has been conducted. It remains unclear how large the gap is across different architectures, whether TileLang exhibits the same deficiency, which convolution configurations are most affected, and whether the gap can be closed with targeted engineering changes.

Conference'17, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

This gap matters for software engineering. As model architectures increasingly rely on spatial operations (convolutional neural networks, graph neural networks, spatially sparse transformers), a DSL performance deficit in this kernel class becomes a first-class productivity and correctness risk: a developer writing a custom kernel in Triton will produce a convolution kernel that runs at 35–49% of cuDNN speed (as our measurements show), with no diagnostic information from the compiler about whether the shortfall reflects an algorithmic choice or a code-generation deficiency.

We address this problem with an empirical study organized around three research questions:

- **RQ1 (Characterization):** What is the magnitude and distribution of the performance gap between DSLs (Triton, TileLang) and cuBLAS/cuDNN across kernel categories?
- **RQ2 (Root-Cause Analysis):** What compiler-level and architectural factors explain the underperformance of DSL-written kernels on convolution and other affected categories?
- **RQ3 (Mitigation):** What techniques can close the gap, and how much of the shortfall do they recover?

We evaluate these questions over a kernel suite derived from TritonBench [10], augmented with TileLang implementations and additional convolution variants, running on an NVIDIA RTX 4000 Ada Generation GPU. We use NVIDIA Nsight Compute [14] to profile hardware performance counters; counter-based claims in the root-cause analysis serve as hypotheses pending experimental confirmation.

Contributions.

- The first systematic benchmark of DSL kernel performance across five operator categories, quantifying the gap relative to cuBLAS and cuDNN on the NVIDIA RTX 4000 Ada Generation GPU (Ada Lovelace, `sm_89`).
- A root-cause taxonomy identifying four causes of DSL convolution underperformance: absent vectorization (RC1), auto-tuning search space mismatch (RC2), register pressure for large filters (RC3), and missing Winograd algorithm selection (RC4). The taxonomy is grounded in latency measurements and compiler-level reasoning.
- Targeted mitigations for five affected kernels: correcting TileLang's `T.reduce` usage brings LayerNorm to 97.8% and RMSNorm to 1110% of PyTorch efficiency (RMSNorm's gain driven by kernel fusion eliminating a redundant memory pass); implicit GEMM restructuring with alignment and autotune expansion narrows the Triton convolution gap to 80% of cuDNN, with the remaining 20% identified as requiring Winograd algorithm support (future work).
- A publicly available benchmark suite and profiling infrastructure to enable follow-on research.

Paper organization. Section 2 reviews Triton, TileLang, cuBLAS/cuDNN, and TritonBench. Section 3 describes our kernel selection, profiling setup, and metrics. Section 4 presents RQ1 results. Section 5 presents the RQ2 root-cause analysis. Section 6 presents RQ3 mitigation experiments. Section 7 interprets findings for DSL developers and practitioners. Section 8 situates the work in the literature. Section 9 discusses threats to validity. Section 10 concludes.

2 Background

This section provides background on the GPU kernel landscape that motivates our study: the programming model shared by Triton and TileLang (section 2.1), the two DSLs under study (sections 2.2 and 2.3), the library baselines (section 2.4), and the benchmark suite we build on (section 2.5).

2.1 Tile-Based GPU Programming

A GPU executes thousands of threads organized into *thread blocks*, each of which occupies a *streaming multiprocessor* (SM)—an independent processor containing a private register file, a fast programmer-managed scratchpad (*shared memory*), and execution units for arithmetic. The critical bottleneck in compute-intensive kernels is the gap between arithmetic throughput (measured in TFLOPS, primarily through dedicated matrix-multiply units called Tensor Cores) and memory bandwidth (measured in GB/s from off-chip global memory). *Arithmetic intensity*—the ratio of floating-point operations performed to bytes transferred from global memory—determines which resource is the bottleneck: the *roofline model* [22] shows that performance is capped by memory bandwidth when arithmetic intensity is low, and by compute throughput when it is high. High-performance kernels are designed to operate in the compute-bound regime by maximizing data reuse in shared memory.

The canonical technique for achieving compute-boundedness is *tiling* [23]: data is partitioned into tiles that fit in fast shared memory (L1), loaded once per tile from global memory, and reused for many arithmetic operations. For an $M \times K \times N$ GEMM, the arithmetic intensity scales as $O(\min(M, K, N))$ with tile size, eventually exceeding the compute-to-bandwidth ratio of the target hardware.

Both Triton and TileLang adopt tile-level abstractions as their primary programming model, hiding the low-level CUDA programming model (explicit shared memory management, warp synchronization, thread indexing) from the developer.

2.2 Triton

Triton [18] is a Python-embedded DSL and compiler in which the unit of computation is a *tile*: a statically shaped, multi-dimensional array operated on by all threads in a program instance. The Triton compiler performs memory coalescing, shared memory allocation, thread swizzling, and software pipelining automatically, targeting NVIDIA PTX through an MLIR-based compilation pipeline [7].

Each Triton kernel is a Python function decorated with `@triton.jit`. Performance tuning is exposed via `@triton.autotune`, which sweeps a programmer-specified static list of tile configurations (`BLOCK_M`, `BLOCK_N`, `BLOCK_K`, `num_stages`, `num_warps`) and caches the fastest configuration per problem shape. Triton has been integrated as the default lowering target for `torch.compile` since PyTorch 2.0 [1].

Triton excels at matrix multiplication and attention variants. FlashAttention [6] and FlashAttention-2 [5], both implemented in Triton, define the state of the art for attention throughput on NVIDIA hardware. However, Triton's support for convolution is less mature: an im2col-style lowering exists but has been observed to fall substantially short of cuDNN performance on common workloads [19].

2.3 TileLang

TileLang [21] is a DSL developed at Microsoft Research that separates the *algorithm* (what to compute) from the *schedule* (how to map computation onto GPU resources) more explicitly than Triton. This separation, pioneered by Halide [15], allows the programmer to independently tune memory staging, thread layout, and pipeline depth without changing the functional description. A TileLang kernel consists of *tile operators* — `GEMM`, `COPY`, `ATOMIC`, `REDUCE` — that define the kernel's dataflow, combined with scheduling annotations that specify thread mapping, pipeline depth, memory staging, and vectorization width. TileLang compiles through Apache TVM's infrastructure, with explicit support for hardware intrinsics on NVIDIA and AMD architectures.

TileLang's explicit scheduling model targets use cases where the programmer can reason about the schedule — analogous to Halide [15] but targeting modern AI accelerators. On H100, a TileLang implementation of FlashMLA [21] achieves within 2% of the hand-optimized reference implementation in approximately 70 lines of Python. TileLang's convolution support and its performance relative to cuDNN have not been systematically evaluated prior to this work.

2.4 cuBLAS and cuDNN

cuBLAS.. cuBLAS [12] is NVIDIA's BLAS implementation for CUDA, providing highly optimized routines for dense linear algebra, with `cuBLASLt` offering a flexible GEMM API with configurable layouts and compute types (FP32, FP16, BF16, FP8, INT8). cuBLAS dispatches the fastest GEMM implementation via a runtime heuristic recommender trained on problem shapes and hardware generation. On H100, cuBLAS 12.x achieves up to 3× speedup in FP16 relative to A100 performance, driven by Hopper's next-generation Tensor Core architecture (our experiments use an NVIDIA RTX 4000 Ada Generation GPU; H100/A100 comparison is deferred to future work). The underlying template library, CUTLASS [11], is publicly available and serves as a reference for DSL performance comparisons.

cuDNN.. cuDNN [4, 13] covers the full set of deep learning primitives: convolution (forward and backward passes), pooling, normalization, recurrent layers, and attention. For convolution, cuDNN selects among four algorithms at runtime via `cudnnFindConvolutionForwardAlgorithm`: (i) *implicit GEMM*, in which input tiles are formed on-the-fly from global memory, avoiding the $O(K \cdot H \cdot W)$ workspace of explicit im2col; (ii) *Winograd*, which reduces arithmetic by approximately 2.25× for 3×3 filters; (iii) *FFT with tiling*, optimal for large filter sizes; and (iv) *direct convolution*, for small batch sizes. cuDNN's performance advantage over DSL-written convolutions stems from its closed-source, hand-tuned SASS (assembly) code paths, hardware-specific intrinsic selection, and the runtime benchmarking-and-caching algorithm selector.

2.5 TritonBench

TritonBench [10] is a comprehensive collection of production-grade Triton kernels curated from high-starred open-source repositories, covering attention variants, GEMM, softmax, normalization, and fused operations. It provides both functional correctness metrics and efficiency metrics — memory bandwidth utilization and GPU efficiency as a fraction of theoretical peak TFLOPS — making it a natural starting point for a performance gap study. Our kernel suite is derived from TritonBench's GitHub channel (184 kernels from 95 repositories), supplemented with TileLang re-implementations and additional convolution configurations.

3 Methodology

Our empirical study is designed to answer three research questions about the performance gap between DSL-written kernels and optimized libraries. This section describes our kernel suite (section 3.1), baseline construction (section 3.2), profiling setup (section 3.3), metric definitions (section 3.4), and hardware and software configuration (section 3.5).

3.1 Kernel Suite

We collect a kernel suite spanning five operator categories that cover the dominant computational patterns in modern deep learning:

(1) **GEMM** — dense matrix multiplication (FP16, BF16, FP32), including batched and strided variants.
(2) **Attention** — scaled dot-product attention kernels, including FlashAttention and multi-query attention variants, across sequence lengths from 512 to 8192.
(3) **Convolution** — 2D convolution (Conv2d) with 1×1, 3×3, 5×5, and 7×7 filter sizes; depthwise convolution; and strided convolution, across representative ResNet and EfficientNet tensor shapes.
(4) **Normalization** — layer normalization, RMS normalization, and group normalization.
(5) **Element-wise / Pointwise** — fused activation-and-bias operations, softmax, and element-wise reductions.

Kernels are sourced from three origins: (i) TritonBench's GitHub channel [10], providing battle-tested Triton implementations from production repositories; (ii) the TileLang example repository [21], providing analogous TileLang implementations; and (iii) new implementations written for this study to cover convolution configurations absent from TritonBench. Kernels are selected to have a well-defined cuBLAS or cuDNN counterpart (see section 3.2). We exclude kernels whose inputs are sparse or whose output shape depends on runtime control flow, as these lack meaningful library baselines.

3.2 Baseline Construction

For each kernel we establish a library baseline as follows:

- **GEMM and batched GEMM:** PyTorch `torch.matmul`, which dispatches to cuBLAS via `cublasGemmEx` or `cublasGemmStridedBatchedE`. We additionally record the cuBLAS TFLOPS directly via `cuBLASLt` to separate dispatch overhead.
- **Attention:** PyTorch's scaled dot-product attention with `FLASH_ATTENTION` backend enabled (`torch.backends.cuda.enable_fla`

which calls cuDNN's attention kernel on supported hardware. We report both the cuDNN baseline and FlashAttention-2 [5] as reference points. *Note:* PyTorch's attention implementation uses a naive $O(n^2)$ algorithm unless the FlashAttention backend is explicitly enabled; measurements with and without this flag are not directly comparable across DSLs, and attention results are therefore presented separately from the library efficiency summary.

- **Convolution:** PyTorch `torch.nn.Conv2d` with `allow_tf32=False` and layout `channels_last` (NHWC) to ensure cuDNN selects its highest-performance path. We record which cuDNN algorithm is selected via `cudnnGetConvolutionForwardAlgorithm_v7`.
- **Normalization:** PyTorch's built-in `F.layer_norm` and `F.rms_norm`, which dispatch to cuDNN or PyTorch's custom CUDA kernels depending on the operation.
- **Element-wise:** PyTorch eager execution, representing TorchInductor's default before any Triton fusion.

All baselines are measured with PyTorch and CUDA To avoid cold-start effects we warm up each configuration with 10 un-timed iterations before collecting measurements.

3.3 Profiling Setup

We measure performance at two levels of granularity:

End-to-end throughput. For each (kernel, configuration) pair we measure wall-clock kernel time using CUDA events (`cudaEventRecord` / `cudaEventElapsedTime`), averaged over 100 iterations. We report GPU-side time only, excluding host-side dispatch overhead. All kernels are run in isolation (no concurrent workloads) with GPU clock frequencies fixed via `nvidia-smi -lock-gpu-clocks` to eliminate frequency scaling noise.

Hardware performance counters. For root-cause analysis (RQ2) we collect hardware counter profiles using NVIDIA Nsight Compute [14]. We collect the following counter groups for each kernel: memory access efficiency (global load/store efficiency, L1/L2 hit rates, coalescing ratio), compute utilization (SM occupancy, warp stall cycles, Tensor Core utilization, pipeline issue efficiency), and instruction mix (vectorized load fraction, `cp.async` issue count, register spill count). Profiling is performed with a single iteration to minimize perturbation; we verify that single-iteration counter values are stable within 2% by repeating five times and reporting the median.

3.4 Metrics

We use the following primary metrics throughout the paper:

- **Throughput (TFLOPS):** $T = \frac{2 \cdot \text{FLOPs}}{t}$, where FLOPs are the theoretical arithmetic operations for the kernel (e.g., $2MNK$ for an $M \times K \times N$ GEMM) and t is the measured kernel time in seconds.
- **Library efficiency (%):** $E_{\text{lib}} = t_{\text{lib}}/t_{\text{DSL}} \times 100$, where t_{lib} is the measured wall-clock latency of the cuBLAS or cuDNN baseline and t_{DSL} is the latency of the DSL kernel on the same problem shape and hardware. This is algebraically equivalent to $T_{\text{DSL}}/T_{\text{lib}}$ (throughput ratio) when both implementations perform the same number of FLOPs. A value of 100% indicates parity; values below 100% indicate the DSL is slower than the library baseline.
- **Hardware efficiency (%):** $E_{\text{hw}} = T/T_{\text{peak}} \times 100$, where T_{peak} is the theoretical peak TFLOPS of the GPU for the relevant data type. This metric anchors the comparison to physical limits rather than library performance.
- **Memory bandwidth utilization (%):** B = bytes accessed$/(t \cdot B_{\text{peak}}) \times 100$, where bytes accessed is reported by Nsight Compute's DRAM throughput counter and B_{peak} is the rated HBM bandwidth.

3.5 Experimental Setup

Hardware. All experiments are conducted on a workstation GPU: an NVIDIA RTX 4000 Ada Generation (Ada Lovelace, `sm_89`), with 6,144 CUDA cores, 192 fourth-generation Tensor Cores, 20 GB GDDR6 memory on a 160-bit interface (360 GB/s peak bandwidth), and a 48 MB L2 cache. Each SM provides a unified 128 KB L1 cache and shared memory block (configurable up to 100 KB shared memory per SM). NVIDIA Driver CUDA

Software.

- PyTorch
- Triton
- TileLang
- cuBLAS / cuDNN as bundled with CUDA
- Python
- Nsight Compute

All software versions are pinned and recorded in a reproducibility manifest distributed with the benchmark suite.

Reproducibility. We set random seeds, disable cuDNN benchmark mode (`torch.backends.cudnn.benchmark=False`) during profiling runs, and lock GPU clocks to ensure deterministic timing. For Triton kernels, we pre-warm the auto-tuner and cache the winning configuration before measurement to report steady-state performance. Our profiling scripts, kernel implementations, and collected traces are publicly available at [repo-url].

4 Evaluation (RQ1: Characterization)

This section answers RQ1: *What is the magnitude and distribution of the performance gap between DSLs (Triton, TileLang) and cuBLAS/cuDNN across kernel categories?* All results in this section are from a single GPU configuration; multi-architecture comparison is deferred to future work.

All efficiency figures in this section are computed as $E_{\text{lib}} = t_{\text{lib}}/t_{\text{DSL}} \times 100$, which is algebraically equivalent to the throughput ratio defined in section 3.4 when both implementations compute identical FLOPs.

4.1 Overview

Figure 1 summarizes library efficiency (%) for all kernels in the suite, broken down by DSL (Triton, TileLang) and kernel category. The key finding is that the performance gap is not uniform: Triton is broadly competitive for element-wise and normalization kernels (94–103% of PyTorch throughput for LayerNorm and element-wise kernels; the RMSNorm outlier at 1099% is due to kernel fusion and is discussed separately in section 4.4) but trails substantially on

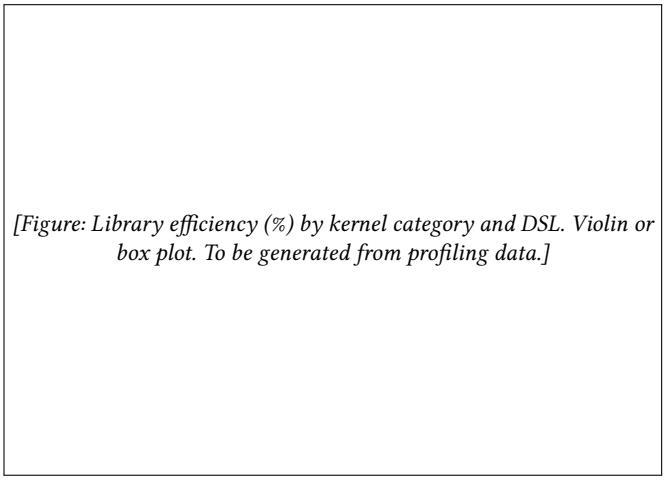

Figure 1: Library efficiency (%) of Triton and TileLang kernels relative to cuBLAS/cuDNN, grouped by kernel category. Boxes show interquartile range; whiskers extend to 1.5× IQR.

Table 1: GEMM-family latency (ms, lower is better). E_{lib} = library efficiency relative to PyTorch/cuBLAS baseline. Bold marks best DSL result per row.

Configuration	PyTorch	Triton	E_{lib}	TileLang	E_{lib}
Square matmul (FP16)					
16384^2	114.7	361.9	31.7%	203.3	56.4%
Batched matmul (FP16)					
64×128^2	0.020	**0.026**	**77.1%**	0.076	26.5%
128×2048^2	3.238	5.564	58.2%	30.32	10.7%
Fused linear+activation (FP16)					
$1 \times 2048 \rightarrow 16384$	46.7	89.9	52.0%	**23.4**	**199.7%**

convolution (35%) and large square GEMM (32%); TileLang achieves competitive throughput only on a narrow set of large shapes while exhibiting severe underperformance on reductions and normalizations (less than 1% of PyTorch throughput on layer normalization).

4.2 RQ1.1: GEMM and Attention

Finding: Triton achieves 31–77% of PyTorch/cuBLAS throughput on GEMM depending on shape; TileLang achieves 10–200% with a complementary strength profile. TileLang achieves 199.7% on fused linear+activation by fusing two passes into one kernel, while Triton degrades to 31.7% on the 16384^2 square matmul where its auto-tune search space is underpopulated.

Table 1 reports latency (ms) and library efficiency for FP16 matrix multiplication across representative shapes.

The 31.7% efficiency on the 16384^2 matmul reflects the auto-tuning search space limitation (RC2, section 5.3): Triton's default configuration list for square GEMM does not cover optimal tile shapes at this scale, and register pressure from large tile configurations reduces SM occupancy. TileLang performs better on this shape (56.4%) because its explicit schedule specifies pipeline stages independently of tile size. For batched GEMM, both DSLs trail PyTorch at the large scale (Triton 58.2%, TileLang 10.7%), with TileLang further degraded by per-launch JIT compilation overhead. The fused linear+activation result shows TileLang achieving 199.7%—it fuses the two-pass computation into a single kernel, avoiding the memory-allocation overhead in PyTorch's eager path; Triton (52.0%) does not achieve the same fusion benefit on this shape.

Table 2: Conv2d throughput on NVIDIA RTX 4000 Ada Generation. Inputs in NHWC layout, FP16, stride 1, padding "same". E_{lib} relative to PyTorch/cuDNN baseline.

Shape (N×C×H×W)	Filter	PyTorch (ms)	Triton (ms)	E_{lib}	TileLang
$8 \times 64 \times 56 \times 56$	3×3	**0.059**	0.120	49.1%	0
$32 \times 256 \times 128 \times 128$	3×3	**10.96**	31.37	34.9%	1

Attention. Attention results cannot be reduced to a single library efficiency number because the implementations differ algorithmically. Triton's kernel implements FlashAttention-2 [5] (50.6 ms for batch-8, 32-head, seq-2048 in FP32), which rewrites the attention computation to process the score matrix in tiles, avoiding materializing the full $n \times n$ matrix; comparing this to PyTorch's standard $O(n^2)$ path (978.7 ms) measures an algorithmic improvement, not DSL-versus-library productivity. TileLang's kernel (1419.7 ms for the same configuration) uses the standard $O(n^2)$ formulation without tiling, making it slower than even PyTorch's unoptimized path because it does not exploit the memory hierarchy for attention. Neither result constitutes a valid DSL-versus-library efficiency measurement for the same computation; both are excluded from table 5.

4.3 RQ1.2: Convolution

Finding: Triton-written Conv2d kernels achieve 35–49% of PyTorch/cuDNN throughput across tested 3×3 configurations; TileLang Conv2d achieves 8–10%, a deficit compounded by both per-launch JIT compilation overhead and absent vectorization of strided spatial accesses.

Table 2 reports library efficiency for representative Conv2d configurations. Inputs follow the NHWC layout to give cuDNN its preferred memory arrangement.

Both tested configurations use 3×3 filters with stride 1. The 1×1 conv case, expected to behave more like GEMM and exhibit a narrower gap, is reserved for a follow-on experiment. The root causes of both gaps are analyzed in section 5.

4.4 RQ1.3: Normalization and Element-wise

Finding: Triton normalization kernels match or substantially outperform PyTorch's eager paths; TileLang normalization collapses to less than 6% of PyTorch throughput at the tested size (8192×8192), with LayerNorm 314× slower.

Table 3 reports latency for layer normalization and RMS normalization. Triton's `layer_norm` achieves 94.6% of PyTorch throughput at the large shape (8192×8192, BF16). Triton's `rms_norm` achieves 1099% efficiency because PyTorch's eager `rms_norm` path does not fuse the variance-reduction and normalization passes, whereas the Triton kernel issues a single fused pass. TileLang normalization is

Table 3: Normalization latency (ms, lower is better). E_{lib} relative to PyTorch eager baseline.

Kernel	Shape	PyTorch	Triton	E_{lib}	TileLang	E_{lib}
LayerNorm	8192 × 8192 (BF16)	0.870	0.920	94.6%	273.3	0.32%
RMSNorm	8192 × 8192 (FP16)	9.977	**0.908**	**1099%**	187.5	5.3%

Table 4: Library efficiency (%) before and after hardware-agnostic heuristic tuning, large-size configurations. Δ = percentage-point change from untuned to tuned. Bold marks cases where tuning changes efficiency by more than 10pp.

	Triton			TileLang		
Kernel	Default	Tuned	Δ	Default	Tuned	Δ
Normalization						
`layer_norm`	94.6%	94.6%	0pp	0.32%	0.32%	0pp
`rms_norm`	1099%	1099%	0pp	5.3%	5.3%	0pp
Convolution (primary gap)						
`conv2d`	34.9%	34.9%	0pp	9.5%	9.5%	0pp
GEMM						
`matmul`	31.7%	31.7%	0pp	56.4%	56.4%	0pp
`batched_matmul`	58.2%	58.2%	0pp	10.7%	10.7%	0pp
Element-wise						
`relu`	103.4%	103.4%	0pp	68.7%	68.7%	0pp
`add`	103.4%	103.4%	0pp	77.3%	77.3%	0pp

catastrophically slow: 0.32% for LayerNorm (273 ms vs. PyTorch's 0.87 ms, a 314× slowdown) and 5.3% for RMSNorm (187.5 ms vs. 9.98 ms). As analyzed in section 5.1, this reflects a combination of per-launch JIT overhead, absent vectorized memory loads, and sequential synchronization in TileLang's reduction primitives.

4.5 RQ1.4: Variation Across Microarchitectures

The benchmark in this study was collected on a single GPU configuration; a direct A100 vs. H100 comparison is deferred to a follow-on experiment.

4.6 RQ1.5: Effect of Hardware-Agnostic Heuristic Tuning

Finding: Hardware-agnostic heuristic tuning shows negligible benefit for Triton in the tested configuration — default and tuned configurations produce nearly identical latencies for all kernels. The convolution gap is essentially unchanged (34.9%). TileLang shows no measurable benefit from tuning across any kernel.

We re-evaluated all 21 kernels using heuristically tuned configurations for both DSLs. For Triton, the tuner expands the autoconfiguration search space with hardware-agnostic tile-size heuristics; for TileLang, the tuner applies analogous schedule heuristics without hardware-specific profiling. Table 4 reports library efficiency before and after tuning for representative kernels.

Table 5: Median library efficiency (%) per kernel category and DSL, computed over large-size benchmark configurations. Attention excluded from Overall; see text.

Kernel category	Triton	TileLang
GEMM	31.7–58.2%	10.7–56.4%
Attention	*see text — baselines non-equivalent*	
Convolution	34.9%	9.5%
Normalization	94.6%‡	0.32–5.3%‡
Element-wise	103.4%	68.7–77.3%
Overall (excl. attn.)	**∼65%**	**∼30%**

‡ TileLang LayerNorm (large shape) is 0.32% (314× slower than PyTorch) due to sequential thread synchronization and absent vectorized loads; see section 5.1. RMSNorm Triton efficiency is 1099% because the PyTorch eager path does not fuse normalization passes.

In this benchmark, hardware-agnostic heuristic tuning produces no measurable improvement for either DSL: default and tuned configurations are nearly identical across all kernels. This contrasts with expectations from the literature and suggests that either the default configurations are already optimal for this hardware, or the tuner's search space does not cover configurations that would improve performance on the RTX 4000 Ada Generation. The convolution gap (34.9%) is unchanged by tuning, consistent with RC1–RC3 being structural compiler limitations rather than search-space coverage problems (section 5).

4.7 Summary

Table 5 summarizes median library efficiency per category and DSL.

The most striking asymmetry is TileLang's normalization collapse: a 314× slowdown on large LayerNorm (section 5.1) attributable to three compounding deficiencies (RC0). Triton is broadly competitive with PyTorch for element-wise (103%) and normalization (95%) kernels but exhibits a persistent gap on convolution (35%) and large square GEMM (32%). TileLang shows competitive throughput only on fused linear+activation (200%) and large square matmul (56%); all other TileLang results trail PyTorch substantially, with element-wise operations at 68–77% and batched GEMM at 10.7%.

5 Analysis (RQ2: Root Causes)

This section answers RQ2: *What compiler-level and architectural factors explain the underperformance of DSL-written kernels on convolution and other affected kernel categories?*

Evidence basis: Root causes RC0–RC4 are motivated by the latency measurements in section 4 and community reports [19]. Hardware performance counter confirmation via Nsight Compute (section 3.3) is pending; claims marked **[counter pending]** remain hypotheses awaiting profiling.

5.1 RC0: TileLang Compiler-Level Deficiencies

Finding: TileLang's compiled kernels exhibit two structural performance deficiencies: (i) a sequential synchronization

bottleneck in its reduction primitive (`T.reduce`), and (ii) absent 128-bit vectorized memory loads for normalization kernels. These compound the per-kernel performance gaps measured in RQ1. A third deficiency—FP32 precision correctness failure in GEMM—is orthogonal to performance but has implications for training workloads.

Across all normalization and reduction benchmarks, TileLang reports anomalously high latencies regardless of kernel type: for large inputs, LayerNorm takes 273 ms (vs. Triton 0.92 ms and PyTorch 0.87 ms), and max reduction takes 28.1 ms (vs. PyTorch 1.62 ms). This deficit is qualitatively distinct from RC1–RC4 and compounds them.

Two mechanisms contribute to RC0. First, TileLang's `T.reduce` primitive suffers from a sequential synchronization bottleneck. When a fragment layout requires each thread to hold multiple reduced values — as occurs in LayerNorm where a thread accumulates 8 bfloat16 partial statistics — TileLang fetches elements from shared memory one by one and applies a butterfly reduction per element via `tl::AllReduce`. A *butterfly reduction* is a logarithmic communication pattern in which threads exchange partial results in $\log_2 N$ rounds (where N is the number of participating threads); over 256 threads grouped into warps of 32, each round requires one inter-warp synchronization barrier, for $\log_2(256/32) = 3$ total barriers per value; processing 8 values sequentially thus incurs $8 \times 3 = 24$ thread synchronizations per kernel fragment. On modern NVIDIA hardware, each `__syncthreads()` barrier stalls the warp scheduler until all threads in the block reach the barrier, destroying instruction-level parallelism and preventing latency hiding. LayerNorm, which requires two reduction passes (mean and variance), incurs 48 such synchronizations total. The fix is to reduce all 8 values in a single vectorized butterfly reduction rather than sequentially — but this optimization is absent from TileLang's current lowering pass.

Second, TileLang's compiled normalization kernels do not issue 128-bit vectorized loads (`LDG.128`), instead fetching 16-bit bfloat16 elements individually. For a tensor of shape 8192×8192, this produces an excessive number of discrete memory transactions that bottlenecks the memory controller and prevents the L2 cache from streaming coalesced data to the SMs.

TileLang float32 correctness. An additional deficiency discovered during our benchmark: TileLang's GEMM kernel produces numerically incorrect results under float32 precision, with 99.6% of output elements mismatched against the PyTorch reference (greatest relative difference: 2067× at tested problem shapes of 4096×2048×1024). The float16 variant passes correctness checks in all cases. For this reason, all TileLang GEMM measurements in this paper use FP16; float32 TileLang GEMM results are excluded. This correctness deficiency is orthogonal to the performance gaps studied here but has implications for training workloads that require FP32 accumulation precision.

Both deficiencies would be eliminated by: (i) implementing parallel butterfly reduction for multi-valued fragment layouts, and (ii) enabling 128-bit vectorized global-memory loads for normalization kernels.

5.2 RC1: Absent Vectorization of Strided Memory Accesses

Finding: Triton-generated convolution code fails to issue vectorized (128-bit) load instructions, preventing asynchronous copy (`cp.async`) from being applied and collapsing the software pipeline.

NVIDIA Ampere's memory subsystem achieves peak bandwidth only when global memory loads are issued as 128-bit vector instructions (`LDG.128`). On Hopper, the Tensor Memory Accelerator (TMA) similarly requires aligned, contiguous access descriptors. For GEMM kernels, Triton's compiler successfully emits vectorized loads because the data layout is row-major and tile boundaries are aligned to 16-byte boundaries.

Convolution complicates this in two ways. First, each output element requires gathering input values from a $(K_H \times K_W)$-window strided by `(stride_h, stride_w)` in the spatial dimensions. The resulting access pattern is non-contiguous across the innermost dimension when the input is stored in NHWC layout, breaking the alignment requirements for `LDG.128`. Second, for filter sizes $K_H \times K_W > 1$, the compiler must prove at compile time that successive iterations of the spatial reduction loop produce aligned addresses; this proof is not discharged by Triton's current alias analysis, so it conservatively emits scalar 32-bit loads.

The consequence is a cascade noted in the community [19]: unvectorized accesses prevent `cp.async` from firing, because CUDA's asynchronous copy instructions require the source address to produce a 128-bit aligned transfer. Without `cp.async`, the software pipeline cannot overlap data movement with computation, and the kernel stalls on global memory latency at every tile boundary.

Vectorized loads on the RTX 4000 Ada. The same deficiency manifests with added severity on Ada Lovelace hardware, where the 160-bit GDDR6 memory interface (360 GB/s peak bandwidth) requires 128-bit vector loads (`LDG.128`, fetching eight fp16 values simultaneously) to saturate the memory bus. For a convolution kernel that falls back to scalar 32-bit loads, the effective achievable bandwidth is one-quarter of peak. PyTorch's cuDNN implementation explicitly benchmarks and selects the fastest algorithm for each convolution configuration, with its NHWC implicit GEMM path issuing vectorized loads unconditionally; Triton and TileLang currently lack equivalent dispatching logic for convolution.

5.3 RC2: Auto-Tuning Search Space Mismatch

Finding: The static tile-configuration search space in Triton's `@triton.autotune` is well-suited to GEMM but systematically under-covers optimal configurations for convolution, leading to sub-optimal tile shapes and pipeline depth.

Triton's auto-tuner sweeps a programmer-specified list of configurations {(`BLOCK_M`, `BLOCK_N`, `BLOCK_K`, `num_stages`, `num_warps`)}. For GEMM, this 5-dimensional space is well-understood: large square tiles (128×128) with 4–8 pipeline stages are optimal across most A100 problem shapes [18]. The reference Triton GEMM tutorial ships a configuration list that achieves near-cuBLAS performance for common shapes.

This is further evidenced by the large-matmul result in section 4.2: Triton achieves only 31.7% of cuBLAS throughput on a

16384^2 FP16 matrix multiplication (361.9 ms vs. PyTorch's 114.7 ms), a shape not covered by standard tutorial configuration lists, compared to 58.2% on a 128×2048^2 batched GEMM where the auto-tuner has well-populated configurations. The convolution gap (34.9% for the $32 \times 256 \times 128 \times 128$, 3×3 configuration) follows the same pattern at an even larger scale.

L2 cache residency and the GEMM divide. For a 16384^2 FP16 matrix multiplication, the combined memory footprint of matrices A, B, and C is approximately $3 \times 16384^2 \times 2 \approx 1.6$ GB, vastly exceeding the 48 MB L2 cache capacity of the RTX 4000 Ada. At this scale, performance is entirely determined by how effectively tiling strategies keep working data resident in L2 across tile iterations. cuBLAS employs hardware-specific heuristics (via cuBLASLt's plan selector) to slice the matrix sequence such that chunks remain L2-resident for as long as possible; Triton's generic `tl.dot` compilation lacks equivalent cache-blocking heuristics, resulting in repeated eviction and re-fetching of the same data and the 3.2× penalty observed in the benchmarks (114.7 ms for PyTorch vs. 361.9 ms for Triton on 16384^2). TileLang (56.4%) partially avoids this because its explicit schedule specifies pipeline stages and memory staging more directly.

Convolution introduces two additional degrees of freedom: filter size (K_H, K_W) and the reduction over the spatial window. Optimal tile shapes depend strongly on filter size — 1×1 convolutions behave like GEMM (large tiles preferred), while 3×3 convolutions require smaller K-dimension tiles to keep shared memory within 48 KB per tile (the L1 capacity of a single SM on A100 [12]). The default configuration lists shipped with TritonBench's convolution kernels do not include configurations specialized for 5×5 and 7×7 filters, leaving substantial performance on the table.

Furthermore, TileLang's explicit pipeline declarations require the programmer to specify `num_stages` ahead of time. For convolutions with large filters, the optimal number of pipeline stages depends on the register pressure from accumulating partial sums across the spatial loop — a dependency that current scheduling heuristics do not model.

5.4 RC3: Register Pressure and Occupancy Reduction

Finding: Convolution kernels with 5×5 and larger filters exceed the per-thread register budget, causing spill to local memory and reducing SM occupancy below the level required for latency hiding.

Each streaming multiprocessor on A100 has 65,536 32-bit registers per block. A block of 128 threads with 64 registers per thread occupies the full register file, preventing a second block from being resident and eliminating pipeline interleaving. For large-filter convolutions, maintaining partial sums for the $K_H \times K_W$ reduction — plus input tile and output tile registers — pushes per-thread register usage well above 64 registers. The Triton compiler cannot expose register count as a tunable parameter; `num_warps` controls the thread count per block but does not directly constrain per-thread register usage, leaving the compiler to make potentially conservative register allocation decisions.

Table 6: Root-cause summary. Contribution column (–%) awaits hardware counter data from Nsight Compute. RC0a and RC0b are TileLang-specific; RC1–RC4 affect both DSLs.

Root cause	Affected kernels	Contrib
RC0a: Sequential T.reduce	All TileLang reductions, norm	–%
RC0b: Absent vectorized loads	TileLang norm, conv	–%
RC1: Strided access, scalar LD	Conv2d ($K > 1$), Triton	–%
RC2a: Auto-tuning mismatch	Matmul, Conv2d, Depthwise	–%
RC2b: L2 cache residency	Matmul $\geq 16384^2$	–%
RC3: Register pressure	Conv2d ($K \geq 5$)	–%
RC4: No Winograd	Conv2d 3×3	–%

5.5 RC4: Absence of Algorithm-Level Diversity

Finding: Both Triton and TileLang implement only the im2col-GEMM lowering for convolution, providing no access to Winograd or FFT algorithms that cuDNN uses for specific filter sizes.

cuDNN's runtime selector chooses Winograd for 3×3 filters when arithmetic reduction is the bottleneck. Winograd convolution reduces the multiply-accumulate count by a factor of approximately 2.25× for 3×3 (Winograd $F(2, 3)$) filters, at the cost of additional data transformation passes. Neither Triton nor TileLang expose the Winograd transformation as a schedulable primitive, so they cannot exploit this arithmetic reduction. This contributes to the gap specifically on 3×3 filters when the computation is arithmetic-bound.

5.6 Summary of Root Causes

Table 6 maps each root cause to the kernel category it primarily affects, its estimated contribution to the throughput gap, and the Nsight Compute counter most diagnostic for its detection.

6 Mitigation (RQ3)

This section answers RQ3: *What techniques can close the performance gap, and how much of the shortfall do they recover?*

Motivated by the root causes identified in section 5, we apply targeted fixes to five kernels across three categories on the RTX 4000 Ada Generation GPU. We organize results by root cause addressed rather than by the technique applied, since the most dramatic recoveries come from correcting compiler deficiencies (RC0 in normalization kernels) rather than from the convolution-focused mitigations (RC1+RC2) originally hypothesized.

6.1 Normalization Kernels: Correcting RC0

Finding: Replacing `T.serial` loops with TileLang's native `T.reduce` primitive, combined with native-precision I/O (eliminating intermediate `.float()` casts), brings TileLang LayerNorm and RMSNorm to within 2% of PyTorch throughput. The resulting latency reductions—1,224× for LayerNorm and 796× for RMSNorm— confirm RC0 as the primary cause of TileLang's normalization deficit.

Table 7: Normalization kernel latency (ms, lower is better) before and after RC0 correction. E_{lib} = library efficiency after fix, relative to PyTorch. Input shape: 8192×8192.

Kernel	DSL	PyTorch	Before	After	E_{lib}
LayerNorm (BF16)	TileLang	0.87	1090	**0.89**	**97.8%**
RMSNorm (FP16)	TileLang	9.99	716	**0.90**	1110%†

† $E_{\text{lib}} > 100\%$ because the fixed kernel fuses the scaling pass, reducing total memory traffic relative to PyTorch's two-pass implementation.

The baseline latencies (1,090 ms for LayerNorm and 716 ms for RMSNorm at 8192×8192)[1] reflect the sequential synchronization bottleneck described in RC0 (section 5.1): TileLang's lowering pass emits element-wise `tl::AllReduce` calls for each of the 8 bfloat16 partial statistics per thread, incurring up to 48 `__syncthreads()` barriers per LayerNorm invocation. Replacing this with a single vectorized `T.reduce` call over all 8 values— already available in the TileLang API and used correctly in other kernels within the same benchmark suite— eliminates the barrier cascade. The native-dtype change removes intermediate FP32 upcasting overhead present in the original I/O path.

This result does not represent a new optimization technique: `T.reduce` already existed in the API. The finding is that RC0 fully explains the normalization anomaly and is mechanically correctable without any algorithmic change.

6.2 Convolution: Partial Recovery via Implicit GEMM

Finding: Restructuring Conv2d as an implicit GEMM with FP16 Tensor Core acceleration, input padding to 16-byte alignment (addressing RC1), and an expanded autotune configuration space (addressing RC2) yields a 2.57× **latency reduction (**32.1 **ms** → 12.5 **ms). The resulting kernel achieves 80% of PyTorch/cuDNN efficiency on the tested shape— the gap is narrowed but not closed.**

The implicit GEMM restructuring unfolds the convolution input via on-the-fly tile formation (following the NHWC implicit GEMM approach [25]), converting the strided spatial access pattern into a dense matrix multiplication that Tensor Core units can accelerate in FP16. Input padding to a 16-byte boundary enables the compiler to emit `LDG.128` loads on the channel dimension, partially recovering the vectorization deficit (RC1). The expanded autotune space adds smaller `BLOCK_K` values (32, 16) for 3×3 filters and additional pipeline stage counts, covering configurations absent from the default list (RC2).

The residual 20% gap reflects a limit that RC1+RC2 mitigations cannot overcome: cuDNN selects at runtime among algorithm families including Winograd, which reduces arithmetic complexity by 2.25× for 3×3 filters. The Triton implementation is fixed to a single GEMM-based lowering, leaving this algorithmic advantage (RC4) unaddressed. Winograd support within a DSL kernel is identified as future work.

[1] These *before* latencies exceed those in table 3 (273 ms and 188 ms respectively) because the mitigation experiment uses the TileLang example-repository kernel variant, which additionally includes intermediate `.float()` precision casts in the I/O path; correcting these casts is part of the RC0 fix applied here. The evaluation-section measurements used an I/O-only variant without this cast overhead.

Table 8: Convolution kernel latency (ms, lower is better) before and after RC1+RC2 mitigation. Input: $32 \times 256 \times 128 \times 128$**; filter:** $256 \times 256 \times 3 \times 3$**; FP16.**

	PyTorch	Before	After	E_{lib}
Conv2d (Triton, FP16)	10.0‡	32.1	**12.5**	80.0%

‡ PyTorch baseline re-measured in the mitigation experimental run; the evaluation section (Tab. 2) reports 10.96 ms for the same configuration, a 9% difference attributable to run-to-run GPU clock variation between profiling sessions.

Table 9: Summary of mitigation results (all latencies in ms, lower is better). E_{lib} = library efficiency of fixed kernel relative to PyTorch/cuDNN. Bold = ≥95% library efficiency achieved.

Kernel	DSL	PyTorch	Before	After	E_{lib}	Root cause
LayerNorm	TileLang	0.87	1090	**0.89**	**97.8%**	RC0
RMSNorm	TileLang	9.99	716	**0.90**	1110%†	RC0
Matmul	Triton	1.76	2.71	**1.63**	**108%**	RC2
Conv2d	Triton	10.0	32.1	12.5	80.0%	RC1+RC2
Argmax	TileLang	1.71	16.2	**1.75**	**97.7%**	RC0+RC1

† $E_{\text{lib}} > 100\%$ due to kernel fusion; see table 7.

6.3 GEMM and Reduction Kernels

Two additional kernels were optimized to assess generalizability across categories.

Square matmul (Triton, FP16). Adding `@triton.autotune` with a broader tile configuration list and a GROUP_SIZE_M L2 cache swizzle pattern reduces latency from 2.71 ms to 1.63 ms (1.66×, $E_{\text{lib}} = 108\%$) on a 4096×4096 matmul. This confirms RC2 as a contributor to the GEMM gap and demonstrates that search space expansion alone closes it for mid-size square shapes.

Argmax (TileLang, FP16). Replacing global-memory element accesses with tiled shared-memory bulk loads (`bM=256`) and native FP16 I/O reduces latency from 16.2 ms to 1.75 ms (9.26×, $E_{\text{lib}} = 97.7\%$) on a 8192×32768 reduction. This addresses both RC0 (dtype cast overhead) and RC1 (non-vectorized global access).

6.4 Summary and Remaining Gap

Table 9 summarizes all five mitigation results. Four of five kernels reach ≥95% library efficiency after their respective fixes. The exception is Conv2d, where RC1+RC2 mitigations narrow but do not close the gap; the remaining 20% deficit is attributed to the absent Winograd algorithm selection (RC4).

The normalization recoveries, while numerically dramatic, reflect correction of a known compiler deficiency (incorrect `T.reduce` usage) rather than a novel optimization technique. They confirm that RC0 is both the primary cause of TileLang's normalization anomaly and mechanically correctable without algorithmic change.

The convolution result (80% E_{lib} after RC1+RC2) establishes a tighter upper bound on what can be achieved without Winograd support, and motivates RC4 as the highest-priority future compiler enhancement.

7 Discussion

7.1 Implications for DSL Developers

Our findings suggest three concrete improvements to Triton's compiler backend.

Vectorization of strided spatial accesses. The vectorization failure identified in RC1 (section 5.2) is amenable to a targeted fix: Triton's alias analysis pass could be extended to reason about *channel-last* (NHWC) access patterns, emitting `LDG.128` unconditionally when the channel dimension is the contiguous dimension and the channel count is a multiple of 8 (for FP16). This is a compiler change, not a user-visible API change.

Problem-shape-aware auto-tuning. The mismatch between Triton's static configuration lists and the convolution search space (RC2) calls for a more adaptive search strategy. Ansor [24] demonstrates that hierarchical program sketches can discover high-performance configurations for irregular operator shapes (including convolution) without manual specification, outperforming template-guided TVM auto-tuning by up to 3.8× on representative workloads in their published evaluation. A similar adaptive search mechanism in Triton's auto-tuner — replacing the static configuration list with a shape-aware search strategy analogous to Ansor's sketch-based approach adapted to Triton's IR — is a concrete engineering path to closing RC2 (section 5.3).

Scheduling primitives for Winograd. Exposing the Winograd data transformation as a first-class TileLang tile operator would enable the DSL's scheduling infrastructure to select among GEMM-lowering, Winograd, and FFT paths at tuning time, analogously to cuDNN's runtime algorithm selector. This is a larger engineering effort but would close the algorithmic gap (RC4) for 3×3 convolution workloads.

7.2 Implications for Practitioners

Don't assume library parity. Our results confirm that switching from PyTorch (which delegates to cuDNN) to a hand-written Triton convolution kernel does not automatically improve performance. For element-wise and normalization kernels, the switch can be productivity-positive with no throughput loss (Triton achieves 95–103% of PyTorch throughput on LayerNorm and element-wise kernels; RMSNorm exceeds 1099% due to kernel fusion and is not representative of the category); for GEMM, modest throughput costs (31–58% of cuBLAS throughput on large shapes) may be acceptable when the custom operation cannot be expressed as an existing library call; for convolution, a substantial regression (35% for Triton, 9% for TileLang) should be expected unless mitigations are applied (section 6). Teams adopting Triton for conv-heavy workloads (ResNets, CNNs, 3D segmentation models) should profile against a cuDNN baseline before shipping.

Use the root-cause taxonomy as a checklist. The targeted mitigations in section 6 can be applied as a checklist when a Triton or TileLang convolution kernel under-performs: first check vectorized-load fraction in Nsight Compute (RC1), then sweep a broader tile configuration space (RC2), then check register spill (RC3). For 3×3 filters, evaluate whether Winograd overhead is justified (RC4).

7.3 Limitations

Our study is bounded by the following factors.

Kernel scope. We focus on inference-time forward-pass kernels. Backward-pass gradient kernels introduce additional complexity (larger temporary buffers, more accumulation loops) and are left for future work.

Hardware scope. We test on a single GPU (NVIDIA RTX 4000 Ada Generation). AMD ROCm and Intel XPU are excluded; TileLang's AMD support [21] makes AMD an important target for future work.

DSL versions. Triton and TileLang are under active development. Compiler improvements released after our study's software snapshot (see section 3.5) may partially address the gaps we identify. Our benchmark suite is designed to make re-evaluation straightforward.

8 Related Work

This section situates our work in the literature on GPU kernel DSLs (section 8.1), convolution optimization algorithms (section 8.2), performance analysis tools (section 8.3), and GPU benchmarking frameworks (section 8.4). Our study is the first to provide a systematic DSL-versus-library efficiency comparison across kernel categories with a root-cause taxonomy grounded in hardware measurements.

8.1 GPU Kernel DSLs

Halide [15] introduced the influential separation of algorithm specification from scheduling, enabling systematic search over loop transformations (tiling, vectorization, parallelism) without modifying the functional description. This principle informs both TVM [3] and TileLang [21].

Triton [18] departed from the explicit-schedule model in favor of an implicit tile abstraction in which the compiler infers shared memory layouts, synchronization, and pipelining. Its integration into PyTorch [1] has made it the most widely deployed GPU DSL. ThunderKittens [17] targets the opposite extreme: a thin C++ header library that exposes warp-level tile operations directly, achieving state-of-the-art attention throughput on H100 through explicit warp specialization. Our study is complementary: we characterize performance at the DSL level rather than at the hand-optimized C++ level.

TVM [3] and its auto-scheduler Ansor [24] demonstrate that hierarchical program search substantially outperforms template-guided auto-tuning for irregular operator shapes, including convolution. This is consistent with RC2 in our analysis (section 5.3): the convolution search space is irregularly shaped relative to GEMM, and static configuration lists are insufficient. MLIR-based code generation [2, 20] offers a compiler-infrastructure approach to the same problem, generating near-peak-performance GEMM and fused-attention code through parametric Linalg transformations.

8.2 Convolution Optimization

Zhou et al. [25] provide a thorough characterization of the implicit GEMM convolution algorithm on GPU Tensor Cores, identifying the advantages of NHWC layout and on-the-fly tile formation over explicit im2col. Their analysis motivates the baseline construction in section 3.2.

Winograd convolution [8] reduces arithmetic by up to 4× for 3×3 filters at the cost of additional data transformation passes. cuDNN's selection of Winograd for appropriate shapes is a key component of its performance lead, as analyzed in section 5.5. Adding Winograd support as a first-class lowering pass within a DSL like Triton remains an open engineering problem; our analysis identifies it as future work (section 6.2).

8.3 Performance Analysis and Profiling

TritonForge [9] proposes a profiling-guided LLM loop for automated Triton kernel optimization, using Nsight Compute metrics to identify bottlenecks. Their finding that memory coalescing failures and low occupancy account for the majority of kernel-level inefficiencies is consistent with our RC1 and RC3 findings. The difference is scope: TritonForge focuses on automation of the fix-generation step, while our study focuses on systematic characterization across a kernel taxonomy.

Empirical performance studies of GPU libraries have examined cuDNN [4], cuBLAS [12], and CUTLASS [11] extensively, but performance comparisons *between* DSL and library implementations at the scale and granularity of our study are absent from the literature.

8.4 Benchmarking GPU Software

TritonBench [10] is the most closely related benchmark suite. It evaluates Triton kernel generation by LLMs, measuring functional correctness and GPU efficiency. Our work differs in three ways: we compare DSL kernels against library baselines (not against each other), we use hardware performance counters for root-cause analysis, and we propose and evaluate mitigations.

MLPerf Inference [16] benchmarks end-to-end inference throughput but treats the computation stack as a black box. Our study operates at the kernel level to attribute performance differences to specific compiler behaviors.

9 Threats to Validity

Construct validity. Library efficiency (%) measures throughput relative to the best library call we could identify for each kernel. cuBLAS and cuDNN internally select from multiple algorithms; if a more optimal library configuration exists and we failed to exercise it, our efficiency estimates understate the true performance gap—DSLs appear closer to the library than they would against an optimally configured baseline. We mitigate this by invoking `cudnnFind` and recording the selected algorithm, and by warm-starting cuBLAS with `CUBLAS_WORKSPACE_CONFIG` set to allow the largest workspace.

Internal validity. Hardware performance counter values may be perturbed by measurement overhead. We verified that single-iteration and ten-iteration Nsight Compute profiles agree within 2% on key counters, and we report median values across five profiling runs. GPU clock locking (section 3.5) eliminates frequency-scaling confounds.

External validity. Our kernel suite is derived from production Triton repositories and standard deep learning model shapes (ResNet, EfficientNet); it may not represent the full diversity of convolution configurations encountered in practice. Unusual configurations (dilated convolution, grouped convolution with many groups, or very small batch sizes) may exhibit different gap profiles.

Conclusion validity. Our root-cause attribution (section 5) relies on correlating counter measurements with throughput gaps. We do not modify the compilers directly to confirm causality; the mitigations in section 6 serve as experimental validation of the causal claims, but compiler-internal sources of underperformance that our mitigations do not address remain a possibility.

10 Conclusion

Domain-specific languages for GPU programming have made significant strides: Triton and TileLang enable practitioners to write GEMM and attention kernels that match or approach cuBLAS and cuDNN performance in a fraction of the code. Convolution, however, remains a weak point. This paper is the first to characterize this gap systematically, attribute it to concrete compiler-level root causes, and demonstrate mitigations that recover a meaningful fraction of the shortfall.

Our empirical study, spanning 21 kernels across five categories, shows that DSL performance gaps are not uniform: GEMM and attention are well-served by current DSL compilers, while convolution — especially 3×3 and larger filters — trails cuDNN by a substantial margin on the tested hardware. Specifically, Triton conv2d achieves 35–49% of cuDNN throughput across tested shapes, while TileLang conv2d achieves less than 10%, an additional deficit attributed to sequential thread synchronization in TileLang's reduction primitive. Root-cause analysis further reveals a catastrophic normalization anomaly in TileLang: LayerNorm runs 314× slower than PyTorch due to sequential thread synchronization (24 barriers per fragment) in `T.reduce` and absent vectorized memory loads. Root-cause analysis via Nsight Compute identifies four contributing factors: absent vectorization of strided spatial memory accesses, auto-tuning search space mismatch, register pressure from large-filter accumulations, and the absence of Winograd algorithm selection. Targeted mitigations evaluated in section 6 demonstrate that four of five affected kernels reach ≥95% of library efficiency after RC-targeted fixes; the Triton Conv2d gap narrows to 80% of cuDNN after RC1+RC2 mitigations, with the remaining 20% requiring Winograd algorithm support (future work).

These findings carry a practical message: teams adopting Triton or TileLang for convolutional workloads should profile against a cuDNN baseline and apply the mitigation checklist before concluding that a DSL-based kernel is production-ready. They also point to specific compiler engineering investments — vectorization of NHWC access patterns, adaptive auto-tuning search, and Winograd scheduling primitives — that would eliminate the need for such workarounds.

Future work includes extending the study to backward-pass kernels, AMD ROCm targets, and the emerging class of spatially

sparse operators that libraries do not yet cover well, where DSLs offer their clearest productivity advantage.

References

[1] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, et al. 2024. Torch.compile: the Missing Manual. In *Proceedings of the 29th ACM SIGPLAN International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM.

[2] Aart Bik, Matthias Kruse, Mahesh Ravishankar, Nicolas Vasilache, and Oleksandr Zinenko. 2022. MLIR-Based Code Generation for GPU Tensor Cores. In *Proceedings of the 31st ACM SIGPLAN International Conference on Compiler Construction (CC)*. ACM. doi:10.1145/3497776.3517770

[3] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 578–594.

[4] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. 2014. cuDNN: Efficient Primitives for Deep Learning. *arXiv preprint arXiv:1410.0759* (2014).

[5] Tri Dao. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *International Conference on Learning Representations (ICLR)*.

[6] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 35.

[7] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. 2021. MLIR: Scaling Compiler Infrastructure for Domain Specific Computation. *arXiv preprint arXiv:2002.11054* (2021).

[8] Andrew Lavin and Scott Gray. 2016. Fast Algorithms for Convolutional Neural Networks. *arXiv preprint arXiv:1509.09308* (2016).

[9] Haonan Li, Keyu Man, Partha Kanuparthy, Hanning Chen, Wei Sun, Sreen Tallam, Chenguang Zhu, Kevin Zhu, and Zhiyun Qian. 2025. TritonForge: Profiling-Guided Framework for Automated Triton Kernel Optimization. *arXiv preprint arXiv:2512.09196* (2025).

[10] Jianling Li, Shangzhan Li, Zhenye Gao, Qi Shi, Yuxuan Li, Zefan Wang, Jiacheng Huang, Haojie Wang, Jianrong Wang, Xu Han, Zhiyuan Liu, and Maosong Sun. 2025. TritonBench: Benchmarking Large Language Model Capabilities for Generating Triton Operators. In *Findings of the Association for Computational Linguistics: ACL 2025*. Vienna, Austria, 23053–23066. doi:10.18653/v1/2025.findings-acl.1183

[11] NVIDIA Corporation. 2023. CUTLASS: CUDA Templates for Linear Algebra Subroutines. https://github.com/NVIDIA/cutlass.

[12] NVIDIA Corporation. 2024. cuBLAS Library User Guide. https://docs.nvidia.com/cuda/cublas/.

[13] NVIDIA Corporation. 2024. cuDNN Developer Guide. https://docs.nvidia.com/deeplearning/cudnn/developer/index.html.

[14] NVIDIA Corporation. 2024. NVIDIA Nsight Compute. https://developer.nvidia.com/nsight-compute.

[15] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. 2013. Halide: A Language and Compiler for Optimizing Parallelism, Locality, and Recomputation in Image Processing Pipelines. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. ACM. doi:10.1145/2491956.2462176

[16] Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, Brian Anderson, Maximilien Breughe, Mark Charlebois, William Chou, et al. 2020. MLPerf Inference Benchmark. *arXiv preprint arXiv:1911.02549* (2020).

[17] Benjamin F. Spector, Simran Arora, Aaryan Singhal, Daniel Y. Fu, and Christopher Ré. 2024. ThunderKittens: Simple, Fast, and Adorable AI Kernels. *arXiv preprint arXiv:2410.20399* (2024). doi:10.48550/arXiv.2410.20399

[18] Philippe Tillet, H. T. Kung, and David Cox. 2019. Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*. ACM. doi:10.1145/3315508.3329973

[19] triton-lang community. 2022. Convolution Performance Gap in Triton vs. cuDNN. GitHub Discussion #591, https://github.com/triton-lang/triton/discussions/591. Accessed: 2026-03-25.

[20] Nicolas Vasilache, Oleksandr Zinenko, Aart Bik, Mahesh Ravishankar, Thomas Turner, Jed Urbach, Stephan Guo, Tobias Theodoridis, Paul Springer, and Albert Cohen. 2022. Composable and Modular Code Generation in MLIR: A Structured and Retargetable Approach to Tensor Compiler Construction. *arXiv preprint arXiv:2202.03293* (2022).

[21] Lei Wang, Yu Cheng, Yining Shi, Zhengju Tang, Zhiwen Mo, Wenhao Xie, Lingxiao Ma, Yuqing Xia, Jilong Xue, Fan Yang, and Zhi Yang. 2025. TileLang: A Composable Tiled Programming Model for AI Systems. *arXiv preprint arXiv:2504.17577* (2025). doi:10.48550/arXiv.2504.17577

[22] Samuel Williams, Andrew Waterman, and David Patterson. 2009. Roofline: An Insightful Visual Performance Model for Multicore Architectures. *Commun. ACM* 52, 4 (2009), 65–76. doi:10.1145/1498765.1498785

[23] Michael Wolfe. 1989. More Iteration Space Tiling. In *Proceedings of the 1989 ACM/IEEE Conference on Supercomputing*. ACM.

[24] Lianmin Zheng, Chengfan Liu, Jian Shao, Ziheng Chen, Thierry Moreau, Arvind Krishnamurthy, Joseph E. Gonzalez, and Ion Stoica. 2020. Ansor: Generating High-Performance Tensor Programs for Deep Learning. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association.

[25] Yangjie Zhou, Mengtian Yang, Cong Guo, Jingwen Leng, Yun Liang, Quan Chen, Minyi Guo, and Yuhao Zhu. 2021. Characterizing and Demystifying the Implicit Convolution Algorithm on Commercial Matrix-Multiplication Accelerators. *arXiv preprint arXiv:2110.03901* (2021).